

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación y Diseño Digital

Carrera de Ingeniería de Software



FORM-FB-01

Fichas Bibliográficas

Fundamentos de SD y Arquitecturas Distribuidas

Autores:

Jhinson Stalyn Aucatoma Celorio.

Martes 12 de mayo del 2026

Código Fuente .tex

Ver archivo en Google Drive

Índice general

1. Fichas Bibliográficas	2
Ficha Bibliográfica #1	2
Ficha Bibliográfica #2	2
Ficha Bibliográfica #3	3
Ficha Bibliográfica #4	4
Ficha Bibliográfica #5	5

Capítulo 1

Fichas Bibliográficas

Ficha Bibliográfica #1

Referencia:

Berenguel Gómez, J. L. (2024). *Desarrollo de aplicaciones web distribuidas* (2.^a ed.). Ediciones Paraninfo.

Resumen:

Un sistema distribuido es un conjunto de nodos de cómputo interconectados mediante una red que, desde el exterior, funciona como una unidad coherente. Su correcta implementación requiere tanto hardware adecuado como software especializado capaz de aprovechar los recursos disponibles. Existen lenguajes de programación con soporte nativo para la distribución, como Java (RMI, threads), Go (goroutines y canales) y Erlang (tolerancia a fallos, paso de mensajes). Además, frameworks como MPI, CUDA, PThreads y OpenMP permiten desarrollar soluciones paralelas y distribuidas. Los sistemas modernos frecuentemente combinan modelos de memoria compartida con comunicación entre nodos, formando arquitecturas híbridas de computación distribuida.

Nota Personal:

Esta fuente ayuda a entender qué es y qué no es un sistema distribuido. Uno de los aportes es que la distribución no depende del hardware sino del software diseñado para coordinarlo. Los lenguajes y frameworks que se presentan en este libro se organizan según su modelo de comunicación, lo que permite entender que existen distintas formas de implementar los sistemas de distribución según el problema. Comparada con la fuente de Cardador (Fuente #2), esta se centra más en los componentes técnicos de construcción, mientras que la siguiente profundiza en las propiedades que el sistema debe garantizar una vez construido.

Ficha Bibliográfica #2

Referencia:

Cardador Cabello, A. L. (2024). *Desarrollo de aplicaciones web distribuidas. IFCD0210*. IC Editorial.

Resumen:

Los sistemas distribuidos son colecciones de ordenadores físicamente separados pero conectados por redes de comunicación, donde cada nodo posee sus propios componentes. Para funcionar correctamente, deben cumplir cuatro características importantes: transparencia (el usuario no percibe la distribución), escalabilidad (crecimiento sin pérdida de rendimiento), fiabilidad y tolerancia a fallos (continuidad operativa ante errores mediante réplicas), y consistencia (estado coherente entre todos los nodos). Estos principios permiten alcanzar objetivos como alto rendimiento, alta disponibilidad, movilidad y ubicuidad. Además, se presenta la replicación de recursos como estrategia para asegurar la disponibilidad y recuperación ante fallos sin interrumpir el servicio.

Nota Personal:

Esta fuente complementa a Berenguel, ya que si la primera responde al *cómo* se construye un sistema distribuido, esta responde al *qué debe cumplir*. Las cuatro propiedades: transparencia, escalabilidad, fiabilidad y consistencia, funcionan como criterios de evaluación aplicables a cualquier sistema distribuido, incluidos los basados en microservicios. La distinción entre fiabilidad y tolerancia a fallos es especialmente relevante porque son conceptos que suelen confundirse. Al analizar las fuentes sobre microservicios, se hace evidente que su diseño responde directamente a estas propiedades: la autonomía de cada servicio y la replicación son mecanismos concretos para cumplirlas.

Ficha Bibliográfica #3

Referencia:

Van Steen, M., & Tanenbaum, A. S. (2025). *Distributed systems* (4.^a ed., versión 4.03x). Maarten van Steen. <https://www.distributed-systems.net/index.php/books/ds4/> ISBN: 978-90-815406-4-3 (versión digital).

Resumen:

En este libro, se menciona sobre una comparativa entre los sistemas distribuidos y centralizados; en ese contexto los sistemas distribuidos son redes de computadoras donde procesos y

recursos están suficientemente distribuidos entre múltiples máquinas, operando de forma transparente como una sola unidad. A diferencia de los sistemas descentralizados (donde la distribución es obligatoria por políticas o confianza) , los distribuidos se expanden por necesidades de rendimiento y dependabilidad. En este contexto, los microservicios emergen como unidades de software independientes que corren como procesos separados en red, derivados del diseño orientado a objetos. Complementariamente, las arquitecturas REST gestionan recursos distribuidos mediante interfaces uniformes y ejecución sin estado, simplificando la composición de servicios en sistemas complejos.

Nota Personal:

Esta fuente aporta la perspectiva más amplia ya que introduce una distinción que las fuentes anteriores no hacen: la diferencia entre sistemas distribuidos y descentralizados, lo que aclara por qué un sistema se distribuye y bajo qué lógica. Además, menciona el origen de los microservicios dentro de la evolución arquitectónica, conectándolos con el diseño orientado a objetos y la filosofía Unix. Esto permite entender los microservicios no como una tendencia aislada sino como una consecuencia lógica de principios ya establecidos, lo que da mayor consistencia al análisis de las Fuentes #4 y #5.

Ficha Bibliográfica #4

Referencia:

Waseem, M., Liang, P., & Shahin, M. (2020). A systematic mapping study on microservices architecture in DevOps. *The Journal of Systems and Software*, 170(110798), 110798. <https://doi.org/10.1016/j.jss.2020.110798>

Resumen:

En este artículo se menciona que la Arquitectura de Microservicios (MSA) organiza aplicaciones como conjuntos de servicios pequeños, independientes y desplegados por separado, comunicados mediante APIs REST o RPC. A diferencia de la Arquitectura Orientada a Servicios (SOA), MSA minimiza el acoplamiento entre servicios y busca que cada servicio funcione con mayor independencia, evitando un control central. Además, se indica que la combinación con DevOps potencia atributos como escalabilidad, despleabilidad y mantenibilidad, aunque en su defecto también introduce desafíos en seguridad y rendimiento por la comunicación frecuente entre servicios. La descomposición mediante Domain-Driven Design, el uso de contenedores, pipelines CI/CD y patrones como Circuit Breaker son las estrategias más aceptadas para implementar microservicios de forma robusta en entornos de desarrollo continuo.

Nota Personal:

Mientras las fuentes anteriores abordan los sistemas distribuidos y microservicios desde lo conceptual y arquitectónico, esta fuente los analiza desde la evidencia empírica acumulada en la literatura. El principal aporte es mostrar qué ocurre cuando los microservicios se implementan en contextos reales, como qué problemas concretos aparecen, qué patrones funcionan y qué desafíos siguen sin solución clara. La distinción detallada entre MSA y SOA complementa lo que Van Steen y Tanenbaum introducen brevemente. También resulta útil que identifique la seguridad como el atributo más afectado negativamente.

Ficha Bibliográfica #5

Referencia:

Blinowski, G., Ojdowska, A., & Przybylek, A. (2022). Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation. *IEEE Access: Practical Innovations, Open Solutions*, 10, 20357–20374. <https://doi.org/10.1109/access.2022.3152803>

Resumen:

Este artículo realiza una comparación entre microservicios y arquitectura monolítica. Los microservicios descomponen aplicaciones en servicios autónomos, independientemente desplegables, que se comunican mediante protocolos ligeros como HTTP/REST. Sus principios clave son responsabilidad única, autonomía y encapsulamiento total detrás de APIs. Frente a la arquitectura monolítica, los microservicios ofrecen mayor tolerancia a fallos, escalabilidad horizontal selectiva y flexibilidad organizacional. Sin embargo, experimentalmente en máquina única el monolito supera al microservicio en rendimiento, debido al overhead de comunicación entre procesos. Los microservicios solo justifican su complejidad cuando el sistema maneja alto tráfico concurrente y requiere escalar componentes específicos de forma independiente.

Nota Personal:

Mientras las fuentes anteriores presentan los microservicios destacando sus ventajas, este estudio demuestra experimentalmente que en sistemas de tráfico moderado el monolito puede superar al microservicio en rendimiento y eficiencia de costo. Esto no invalida lo planteado en las otras fuentes, sino que lo contextualiza, ya que los beneficios de los microservicios dependen directamente de la escala del sistema. Complementado con los hallazgos de Waseem et al. sobre los desafíos reales de implementación, queda claro que adoptar microservicios debe ser una decisión basada en necesidades concretas y no en tendencias.